

Generierung von Task Graphen

Anja Graf



Lehrstuhl für Embedded Systems
Institut für Informatik
Universität Augsburg

26. August 2025

Zielsetzung des Forschungsmoduls

Funktionsweise

- Graphgenerierung mit Erdős–Rényi Modell

- Taskparameter

- Uniprocessor Scheduling

- Beispiel Graphgenerierung und Parametervergabe

- Utilization

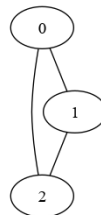
 - UUnifast Algorithmus

 - Dirichlet Rescale Algorithmus

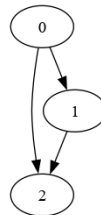
- Beispiel Graphgenerierung mit DRS Utilization



- Generieren eines Taskgraphen mit randomisierter Graphstruktur
 - Knoten sind Tasks, Kanten sind Abhängigkeiten
 - Graph ist Directed und Acyclic (DAG)

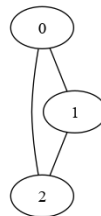


Ungerichteter, zyklischer Graph

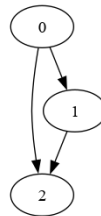


Gerichteter, azyklischer Graph

- ▶ Generieren eines Taskgraphen mit randomisierter Graphstruktur
 - ▶ Knoten sind Tasks, Kanten sind Abhängigkeiten
 - ▶ Graph ist Directed und Acyclic (DAG)
- ▶ Command Line Interface für Benutzerschnittstelle
 - ▶ Übergabeparamter wie Knoten- oder Kantenanzahl

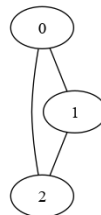


Ungerichteter, zyklischer Graph
Mit 3 Knoten, 3 Kanten

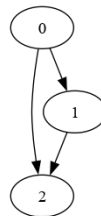


Gerichteter, azyklischer Graph
Mit 3 Knoten, 3 Kanten

- ▶ Generieren eines Taskgraphen mit randomisierter Graphstruktur
 - ▶ Knoten sind Tasks,
Kanten sind Abhängigkeiten
 - ▶ Graph ist Directed und Acyclic (DAG)
- ▶ Command Line Interface für Benutzerschnittstelle
 - ▶ Übergabeparamter wie Knoten- oder Kantenanzahl
- ▶ Output in Form von generierten Dateien



Ungerichteter, zyklischer Graph
Mit 3 Knoten, 3 Kanten



Gerichteter, azyklischer Graph
Mit 3 Knoten, 3 Kanten

Zwei Varianten des Modells:



Zwei Varianten des Modells:

$G(n, p)$

n = Anzahl an Knoten

p = Wahrscheinlichkeit für Kanten

- ▶ Iteriere über geshuffelte Knotenvektoren, um Knoten i, j für jede mögliche Kante $e = \{i, j\}$ zu erhalten ($i \neq j$)
- ▶ Füge diese Kante e mit Wahrscheinlichkeit p hinzu



Zwei Varianten des Modells:

$G(n, p)$

n = Anzahl an Knoten

p = Wahrscheinlichkeit für Kanten

- ▶ Iteriere über geschuffelte Knotenvektoren, um Knoten i, j für jede mögliche Kante $e = \{i, j\}$ zu erhalten ($i \neq j$)
- ▶ Füge diese Kante e mit Wahrscheinlichkeit p hinzu

$G(n, m)$

n = Anzahl an Knoten

m = Anzahl an Kanten

- ▶ Wähle zwei zufällige unterschiedliche Knoten aus
- ▶ Füge Kante zwischen Knoten hinzu, falls neue Kante
- ▶ Iteriere, bis Anzahl m erreicht



Zwei Varianten des Modells:

$G(n, p)$

n = Anzahl an Knoten

p = Wahrscheinlichkeit für Kanten

- ▶ Iteriere über geschuffelte Knotenvektoren, um Knoten i, j für jede mögliche Kante $e = \{i, j\}$ zu erhalten ($i \neq j$)
- ▶ Füge diese Kante e mit Wahrscheinlichkeit p hinzu

$G(n, m)$

n = Anzahl an Knoten

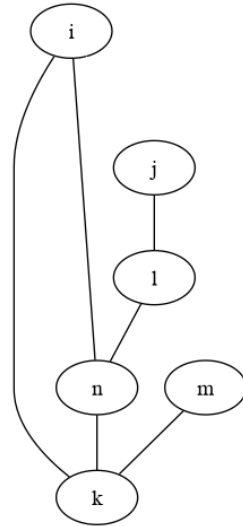
m = Anzahl an Kanten

- ▶ Wähle zwei zufällige unterschiedliche Knoten aus
- ▶ Füge Kante zwischen Knoten hinzu, falls neue Kante
- ▶ Iteriere, bis Anzahl m erreicht

Insgesamt sind beide Varianten noch ungerichtet und prüfen nicht auf Kreisfreiheit!

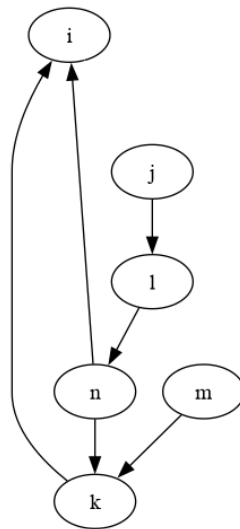


Daher:



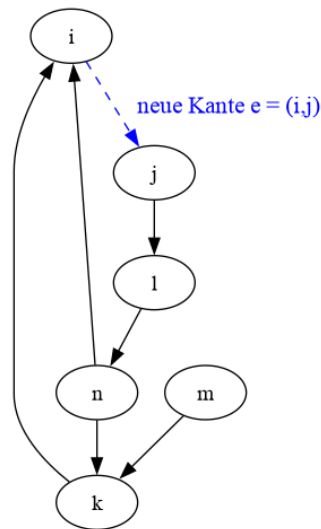
Daher:

1. Füge gerichtete, statt ungerichtete Kanten ein



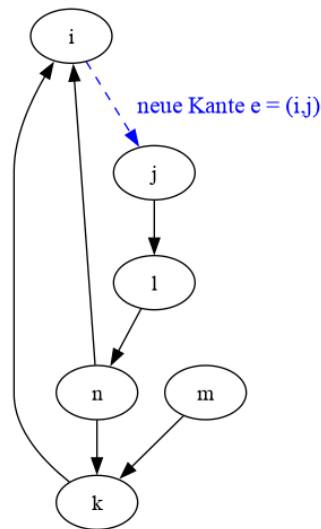
Daher:

1. Füge gerichtete, statt ungerichtete Kanten ein
2. Prüfe beim Hinzufügen einer neuen Kante $e = (i, j)$, ob (i, j) oder (j, i) bereits existiert
 - Falls ja darf e nicht eingefügt werden



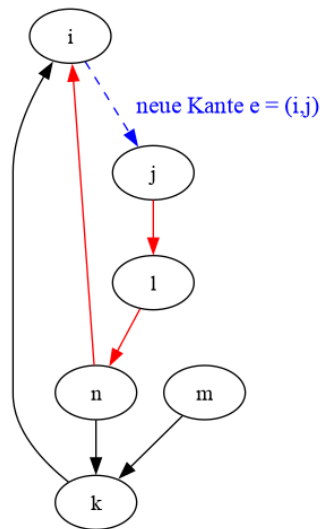
Daher:

1. Füge gerichtete, statt ungerichtete Kanten ein
2. Prüfe beim Hinzufügen einer neuen Kante $e = (i, j)$, ob (i, j) oder (j, i) bereits existiert
 - Falls ja darf e nicht eingefügt werden
3. Prüfe, ob es bereits Pfad j nach i gibt mittels rekursiven Tiefensuche



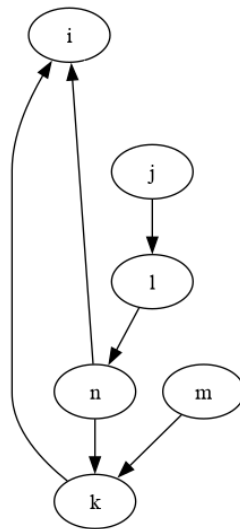
Daher:

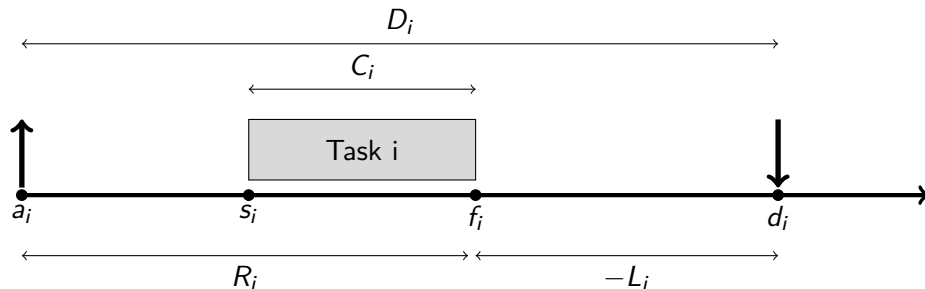
1. Füge gerichtete, statt ungerichtete Kanten ein
2. Prüfe beim Hinzufügen einer neuen Kante $e = (i, j)$, ob (i, j) oder (j, i) bereits existiert
 - Falls ja darf e nicht eingefügt werden
3. Prüfe, ob es bereits Pfad j nach i gibt mittels rekursiven Tiefensuche
 - Falls ja darf e nicht eingefügt werden



Daher:

1. Füge gerichtete, statt ungerichtete Kanten ein
 2. Prüfe beim Hinzufügen einer neuen Kante $e = (i, j)$, ob (i, j) oder (j, i) bereits existiert
 - Falls ja darf e nicht eingefügt werden
 3. Prüfe, ob es bereits Pfad j nach i gibt mittels rekursiven Tiefensuche
 - Falls ja darf e nicht eingefügt werden
- Graph ist Directed und Acyclic ✓





Arrival/Release time a_i

Start time s_i

Computation time C_i

Finishing time f_i

Priority, set to i

Absolute Deadline d_i

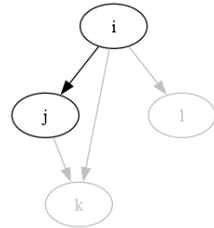
Relative Deadline $D_i = d_i - a_i$

Response time $R_i = f_i - a_i$

Lateness $L_i = f_i - d_i$

Ziel: Ausführungsreihenfolge finden

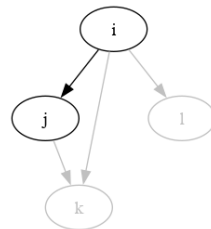
- Berücksichtige dabei Abhängigkeiten



Task i muss vor Task j
ausgeführt werden

Ziel: Ausführungsreihenfolge finden

- ▶ Berücksichtige dabei Abhängigkeiten
- ▶ Generiere zu Beginn Hash Map
 - ▶ Keys: Knoten bzw. Task
 - ▶ Values: Jeweils eine Liste aller Vorgänger



Task i muss vor Task j
ausgeführt werden

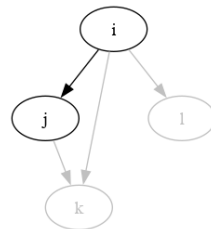
Key	Value
i	
j	i
k	i, j
l	i

Predecessor Hash
Map zu Graph oben



Ziel: Ausführungsreihenfolge finden

- ▶ Berücksichtige dabei Abhängigkeiten
- ▶ Generiere zu Beginn Hash Map
 - ▶ Keys: Knoten bzw. Task
 - ▶ Values: Jeweils eine Liste aller Vorgänger
- ▶ Füge iterativ bereitstehende Tasks mit der höchsten Priorität in den Schedule ein
 - ▶ Task ist bereit/ready, falls vorher keine abhängigen Tasks mehr ausgeführt werden müssen



Task i muss vor Task j ausgeführt werden

Key	Value
i	
j	i
k	i, j
l	i

Predecessor Hash Map zu Graph oben



```
schedule  $\leftarrow$  [], ready  $\leftarrow$  {}, pre  $\leftarrow$  get_predecessor_hash_map()  
forall key, value  $\in$  pre : value =  $\emptyset$  do  
  | ready.add(key)  
end  
// throw error if ready =  $\emptyset$ 
```



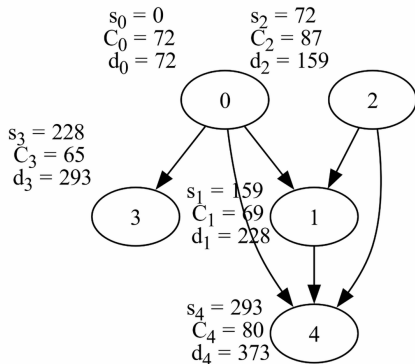
```
schedule  $\leftarrow$  [], ready  $\leftarrow$  {}, pre  $\leftarrow$  get_predecessor_hash_map()  
forall key, value  $\in$  pre : value =  $\emptyset$  do  
  | ready.add(key)  
end  
// throw error if ready =  $\emptyset$   
while ready  $\neq$   $\emptyset$  do  
  | highest_priority  $\leftarrow$  ready.get_min()  
  | ready.remove(highest_priority)  
  | pre.remove(highest_priority)  
  | schedule.append(highest_priority)  
  
  |  
  
end  
return schedule
```



```
schedule  $\leftarrow$  [], ready  $\leftarrow$  {}, pre  $\leftarrow$  get_predecessor_hash_map()
forall key, value  $\in$  pre : value =  $\emptyset$  do
    | ready.add(key)
end
// throw error if ready =  $\emptyset$ 
while ready  $\neq \emptyset$  do
    | highest_priority  $\leftarrow$  ready.get_min()
    | ready.remove(highest_priority)
    | pre.remove(highest_priority)
    | schedule.append(highest_priority)
    | forall key, value  $\in$  pre : highest_priority  $\in$  value do
        | value  $\leftarrow$  value \ highest_priority
        | if value =  $\emptyset$  then
            | ready.add(key)
        | end
    | end
end
return schedule
```



- Beispielhafter Output des Programms bei Aufruf mit Parametern *-nodes 5 -edges 6* und Auswahl des $G(n,m)$ Erdős-Rényi Modell



Task Parameter

Computation time C_i is set to random value in $\{10..100\}$

Arrival/Release time a_i is set to 0 for all i

Absolute Deadline d_i is set to $s_i + C_i$

Relative Deadline $D_i = d_i - a_i = d_i$

Start time s_i

Finishing time f_i is set to d_i

Response time $R_i = f_i - a_i = f_i$

Lateness $L_i = f_i - d_i = 0$

Priority of task is defined by its number

Schedule: [0, 2, 1, 3, 4]

- ▶ Weitere Anforderung:
Berücksichtigung der Auslastung von Tasks



- ▶ Weitere Anforderung:
Berücksichtigung der Auslastung von Tasks
- ▶ Utilization für Task i definiert als $U_i = \frac{C_i}{T_i}$
 - ▶ dabei T_i Periode, d.h. fester Zeitabstand, in welchem die Task i wiederholt ausgeführt werden soll



- ▶ Weitere Anforderung:
Berücksichtigung der Auslastung von Tasks
- ▶ Utilization für Task i definiert als $U_i = \frac{C_i}{T_i}$
 - ▶ dabei T_i Periode, d.h. fester Zeitabstand, in welchem die Task i wiederholt ausgeführt werden soll
- ▶ Gesamtauslastung des Prozessors $U = \sum_{i=1}^n U_i = \sum_{i=1}^n \frac{C_i}{T_i}$
 - ▶ „Wie viel Prozessorzeit pro Zeiteinheit benötigen alle Tasks zusammen?“
 - ▶ Wenn $U > 1$: Prozessor ist überlastet und Tasks sind nicht schedulable/echtzeitfähig



- ▶ Weitere Anforderung:
Berücksichtigung der Auslastung von Tasks
- ▶ Utilization für Task i definiert als $U_i = \frac{C_i}{T_i}$
 - ▶ dabei T_i Periode, d.h. fester Zeitabstand, in welchem die Task i wiederholt ausgeführt werden soll
- ▶ Gesamtauslastung des Prozessors $U = \sum_{i=1}^n U_i = \sum_{i=1}^n \frac{C_i}{T_i}$
 - ▶ „Wie viel Prozessorzeit pro Zeiteinheit benötigen alle Tasks zusammen?“
 - ▶ Wenn $U > 1$: Prozessor ist überlastet und Tasks sind nicht schedulable/echtzeitfähig
- ▶ Im Programm kann zwischen UUnifast und Drs Algorithmus gewählt werden
 - ▶ Input: Knotenanzahl n und Gesamtauslastung U
 - ▶ Output: Vektor $U_1, U_2, \dots, U_n$
 - ▶ Die entsprechenden Werte werden für jede Task i angepasst, damit das Verhältnis mit U_i übereinstimmt



- ▶ ermöglicht Uniforme Verteilung der Gesamtauslastung U in $O(n)$ Komplexität
- ▶ Funktionsweise:

```
sum_u  $\leftarrow$  U
forall  $i \in \{1, \dots, n-1\}$  do
    next_sum_u  $\leftarrow$  sum_u  $\cdot$  rand $^{\frac{1}{n-i}}$ 
    U_i  $\leftarrow$  sum_u - next_sum_u
    sum_u  $\leftarrow$  next_sum_u
end
U_n  $\leftarrow$  sum_u
```

- ▶ Weiterführende Details dazu u.a. in
 - ▶ 'Biasing Effects in Schedulability Measures' von Bini, Buttazzom, Seite 6f.
 - ▶ 'Techniques For The Synthesis Of Multiprocessor Tasksets' von Emberson, Stafford, Davis



```
fn uunifast(&self, u: f64) -> Vec<f64> {  
    assert!(u <= 1.0 && u >= 0.0,  
        "The processor utilization should be value between 0 and 1");  
    let mut sum_u = u;  
    let mut rng = rand::thread_rng();  
    let mut vect_u: Vec<f64> = Vec::new();  
    for i in 0..self.nodes.len() - 1 {  
        let next_sum_u = sum_u * rng.r#gen::<f64>()  
            .powf(1.0 / (self.nodes.len() - i) as f64);  
        vect_u.push(sum_u - next_sum_u);  
        sum_u = next_sum_u;  
    }  
    vect_u.push(sum_u);  
    assert!(u - vect_u.iter().sum::<f64>().abs() < 1e-6);  
    vect_u  
}
```



Constraints für generierte Graphen

Gilt $\forall i \in \{1..n\}$, n = Anzahl der Tasks, i = Index der Task im Schedule

$$C_i \in \{10..100\}$$

$$d_i \geq C_i + s_i$$

$s_i \geq d_{i-1}$, wobei $s_1 = 0$
Der Einfachheit halber $s_i = d_{i-1}$

$U_i = \frac{C_i}{T_i}$
Der Einfachheit halber $T_i = d_i$



Constraints für generierte Graphen

Gilt $\forall i \in \{1..n\}$, n = Anzahl der Tasks, i = Index der Task im Schedule

$$C_i \in \{10..100\}$$

$$d_i \geq C_i + s_i$$

$$s_i \geq d_{i-1}, \text{ wobei } s_1 = 0$$

Der Einfachheit halber $s_i = d_{i-1}$

$$U_i = \frac{C_i}{T_i}$$

Der Einfachheit halber $T_i = d_i$

Es kann durch bestimmte Inputs unmöglich sein, alle Constraints zu erfüllen!

- ▶ z.B. Wenn C_i neu gesetzt wird: kleines $U_i \rightarrow$ kleines $C_i \rightarrow$ kann $C_i \geq 10$ verletzen
- ▶ z.B. Wenn d_i neu gesetzt wird: großes $U_i \rightarrow$ kleines $d_i \rightarrow$ kann $< C_i + s_i$ sein



- ▶ Verbesserung durch DRS Algorithmus
- ▶ Erlaubt zusätzliche Vektoren für untere und obere Schranken
- ▶ Verwendet iteratives Rescaling solange Elemente außerhalb liegen



- ▶ Verbesserung durch DRS Algorithmus
- ▶ Erlaubt zusätzliche Vektoren für untere und obere Schranken
- ▶ Verwendet iteratives Rescaling solange Elemente außerhalb liegen
- ▶ Setze für jedes U_i mit $i \in \{1, \dots, n\}$:
 - ▶ Lower bound $lb_i = \frac{10}{d_i}$
 - ▶ Upper bound $ub_i = \frac{C_i}{d_i}$ (falls vorher Parametervergabe)

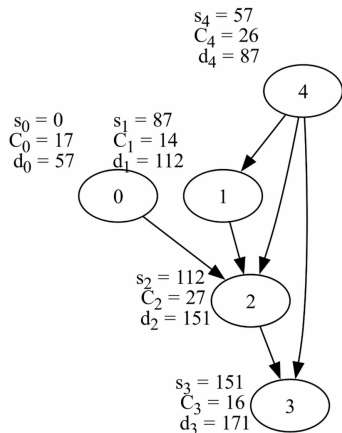


- ▶ Verbesserung durch DRS Algorithmus
- ▶ Erlaubt zusätzliche Vektoren für untere und obere Schranken
- ▶ Verwendet iteratives Rescaling solange Elemente außerhalb liegen
- ▶ Setze für jedes U_i mit $i \in \{1, \dots, n\}$:
 - ▶ Lower bound $lb_i = \frac{10}{d_i}$
 - ▶ Upper bound $ub_i = \frac{C_i}{d_i}$ (falls vorher Parametervergabe)
- ▶ Prüfe vorher, ob $U \geq \sum_{i=1}^n lb_i$ bzw. $U \leq \sum_{i=1}^n ub_i$ und $lb_i \leq ub_i$ (Nutzereingaben möglich)



- ▶ Verbesserung durch DRS Algorithmus
- ▶ Erlaubt zusätzliche Vektoren für untere und obere Schranken
- ▶ Verwendet iteratives Rescaling solange Elemente außerhalb liegen
- ▶ Setze für jedes U_i mit $i \in \{1, \dots, n\}$:
 - ▶ Lower bound $lb_i = \frac{10}{d_i}$
 - ▶ Upper bound $ub_i = \frac{C_i}{d_i}$ (falls vorher Parametervergabe)
- ▶ Prüfe vorher, ob $U \geq \sum_{i=1}^n lb_i$ bzw. $U \leq \sum_{i=1}^n ub_i$ und $lb_i \leq ub_i$ (Nutzereingaben möglich)
- ▶ Ergebnis: erzeugte Task Graphen erfüllen Constraints ✓





- Beispielhafter Output des Programms bei Aufruf mit Parametern ***-nodes 5 -edges 6 -drs 1 -csv parameter.csv*** und Auswahl des $G(n,m)$ Erdős-Rényi Modell

i	s_i	C_i	d_i	D_i	f_i	R_i	L_i	a_i	U_i
0	0	17	57	57	17	17	-40	0	0.298245614035088
1	87	14	112	112	101	101	-11	0	0.125
2	112	27	151	151	139	139	-12	0	0.178807947019868
3	151	16	171	171	167	167	-4	0	0.0935672514619883
4	57	26	87	87	83	83	-4	0	0.298850574712644

Optionales Parameter File aller Tasks

